

Star Wars Tennis

Jean Israel Garces Escobar y Mateo Arbeláez Cardona

Facultad de ingeniería, Universidad de Antioquia

Informática II

Aníbal Guerra y Augusto Salazar

5 Junio de 2026

Introducción

Star Wars Tennis es un videojuego de escritorio desarrollado en C++ utilizando el framework Qt, presentado como proyecto final del curso Informática II de la Facultad de Ingeniería de la Universidad de Antioquia. El proyecto integra los fundamentos teóricos del curso en un producto funcional e interactivo, aplicando programación orientada a objetos, manejo de memoria dinámica y diseño de interfaces gráficas mediante las bibliotecas Qt Widgets y Qt Multimedia.

El desarrollo surgió como respuesta a la necesidad de consolidar, en un único entregable, los tres ejes centrales trabajados a lo largo del semestre. La temática del universo Star Wars fue seleccionada por su reconocimiento cultural y su riqueza visual, permitiendo construir dos modos de juego con identidades propias: un primer nivel inspirado en el clásico Pong, ambientado en la Batalla de Yavin con naves espaciales (vista cenital), y un segundo nivel de acción lateral donde el jugador controla a un Jedi que debe deflectar los disparos de un AT-AT Imperial usando su sable de luz (vista lateral con físicas parabólicas). Esta dualidad permitió explorar mecánicas de juego distintas dentro de una misma arquitectura de software.

El público al que va dirigido el juego es amplio: por un lado, jugadores casuales con interés en la cultura pop de Star Wars que busquen una experiencia accesible y entretenida; por otro, la comunidad académica de la Universidad de Antioquia, para quienes el proyecto sirve como

ejemplo de las capacidades que puede alcanzar un desarrollo en C++ orientado a objetos al nivel de un curso introductorio de programación.

Planteamiento del problema

El desarrollo de software con interfaces gráficas representa uno de los retos más completos para un estudiante de programación, ya que exige no solo dominar la lógica del lenguaje, sino también comprender cómo los distintos componentes de un sistema interactúan en tiempo real.

Star Wars Tennis surge como respuesta a esa necesidad: un proyecto que justifique el uso de herencia, polimorfismo y encapsulamiento no como conceptos abstractos, sino como soluciones concretas a problemas reales de diseño. La existencia de dos modos de juego con comportamientos distintos pero compartiendo una misma jerarquía de clases obligó a tomar decisiones de arquitectura no triviales, como unificar al jugador y al enemigo en clases reutilizables mediante modo dual, o centralizar el control del juego en una ventana principal que gestiona múltiples estados y temporizadores simultáneamente.

Adicionalmente, la incorporación de un agente inteligente en el segundo nivel (modo Jedi) planteó el problema de diseñar un oponente que no solo reaccione al estado del juego, sino que mejore su comportamiento con cada intercambio. Esto añadió una capa de complejidad algorítmica que va más allá de la simple IA reactiva, haciendo necesario implementar un ciclo de percepción, razonamiento, acción y aprendizaje dentro de las restricciones

propias de un juego en tiempo real. Tal como se detalla en la sección 4 del documento, el agente almacena en un `map<int,float>` memoria el error de predicción cometido en cada intercambio, y en el siguiente utiliza ese dato para corregir su estimación de la trayectoria parabólica de la bola de fuego Sith, mejorando progresivamente su capacidad de esquivar.

Definición general y objetivos

Star Wars Tennis es un videojuego de escritorio de un jugador, desarrollado en C++ con Qt, que ofrece dos modos de juego diferenciados por mecánica y ambientación. El alcance funcional del proyecto abarca un menú principal interactivo con selección de dificultad, dos modos de juego completamente implementados, un sistema de sonido con música y efectos, una pantalla de instrucciones, y transiciones entre estados del juego con imágenes de victoria y derrota.

El primer modo, denominado *Tenis Espacial*, recrea la mecánica del juego clásico *Pong* en una ambientación espacial inspirada en la Batalla de Yavin (vista cenital). El jugador controla un X-Wing que se mueve verticalmente a lo largo del lado izquierdo de la pantalla mediante las teclas de dirección, y debe devolver un torpedo protón hacia el lado del Imperio, enfrentándose a un TIE Fighter controlado por la IA. El sistema de puntuación implementa las reglas reales del tenis, incluyendo *games*, *sets*, *deuce*, *ventaja* y *tie-break*. En el centro del campo, un campo de fuerza se desplaza verticalmente de arriba abajo e introduce desvíos aleatorios en la trayectoria del proyectil al impactar contra él, simulando una colisión elástica con desvío aleatorio controlado, lo que añade un elemento de impredecibilidad

en la partida.

El segundo modo, denominado *Modo Jedi*, es un juego de acción lateral (vista lateral fija) donde el jugador controla a Luke Skywalker, quien debe moverse horizontalmente por el suelo, saltar y activar el swing de su sable de luz para enviar las bolas de fuego Sith lanzadas por un AT-AT Imperial. El movimiento del sable se modela como una rotación con ángulo variable alrededor del cuerpo del Jedi, y el golpe es exitoso solo si la bola se encuentra dentro del arco de impacto en el momento exacto del swing. El objetivo es reducir la barra de vida del AT-AT a cero antes de que se agote el tiempo o el jugador pierda todas sus vidas. A lo largo de la partida aparecen power-ups (incremento de daño y congelación) y penalidades (lentitud y velocidad extrema) de forma aleatoria en la pantalla, que el jugador puede recoger al la pelota pasar sobre ellos y que modifican temporalmente las condiciones del juego.

Los objetivos principales del proyecto son tres. Primero, aplicar los principios de la programación orientada a objetos mediante una jerarquía de clases coherente que permita reutilizar componentes entre ambos modos de juego. Segundo, gestionar correctamente la memoria dinámica en un entorno de tiempo real, donde los objetos del juego se crean y destruyen continuamente durante la ejecución. Tercero, construir una interfaz gráfica funcional y visualmente atractiva usando Qt, que responda de forma fluida a las entradas del usuario y mantenga un rendimiento estable durante toda la partida.

Especificación de requerimientos

Requisitos funcionales

El sistema debe ofrecer un menú principal desde el cual el jugador pueda seleccionar entre el modo Tenis Espacial en dificultad fácil o difícil, el modo Jedi en tres niveles de dificultad, o consultar la pantalla de instrucciones. Desde cualquier modo de juego debe ser posible regresar al menú principal presionando la tecla Escape.

El modo Tenis (vista cenital) debe permitir al jugador mover su nave verticalmente a lo largo del lado izquierdo de la pantalla mediante las teclas de dirección, detectar colisiones entre el proyectil y los personajes, aplicar un ángulo de salida variable según el punto de impacto (generando trayectorias más cerradas o más abiertas), registrar puntos cuando el proyectil sale por alguno de los bordes laterales, y determinar un ganador siguiendo las reglas completas del tenis con games, sets, deuce, ventaja y tie-break. El obstáculo central (campo de fuerza) debe desplazarse verticalmente de arriba abajo e introducir desvíos aleatorios al proyectil cuando colisione con él, simulando una colisión elástica con desvío aleatorio controlado.

El modo Jedi (vista lateral fija) debe permitir al jugador moverse horizontalmente por el suelo, saltar y activar el swing del sable de luz, el cual ejecuta un movimiento circular alrededor del cuerpo del Jedi calculado frame a frame. El AT-AT debe disparar bolas de fuego Sith con trayectoria parabólica afectada por la gravedad del entorno, determinada por las ecuaciones de la cinemática de proyectil con componentes horizontal y

vertical independientes. Solo las bolas devueltas por el jugador con el sable (es decir, aquellas que se encuentran dentro del arco de rotación del sable en el momento del swing) pueden dañar al AT-AT. El juego debe generar power-ups (incremento de daño y congelación) y penalidades (lentitud y velocidad extrema) de forma aleatoria en la pantalla, que el jugador puede recoger al pasar sobre ellos. Adicionalmente, debe mostrar una barra de vida del AT-AT, un contador de vidas del jugador y un temporizador de cuenta regresiva que añade presión al jugador. El agente inteligente debe mejorar su capacidad de predicción con cada intercambio mediante un mecanismo de aprendizaje basado en memoria (almacenando el error de predicción en un `map<int,float>` memoria), haciendo que el nivel se vuelva progresivamente más difícil a medida que avanza la partida.

El sistema debe reproducir música de fondo diferenciada para el menú y el modo Tenis, modo Jedi y así como efectos de sonido para golpes, puntos, etc

Requisitos no funcionales

El juego debe mantener una tasa de actualización estable mediante un temporizador principal de 16 milisegundos, equivalente a aproximadamente 60 frames por segundo. La interfaz debe responder de forma inmediata a las entradas del teclado sin retardo perceptible. Los sprites y fondos deben cargarse desde el sistema de recursos de Qt para garantizar portabilidad del ejecutable. El código debe estar organizado en clases con responsabilidades claramente delimitadas, siguiendo los principios de encapsulamiento y separación de concerns.

Restricciones

El proyecto fue desarrollado en C++17 con Qt 6, usando CMake como sistema de construcción. No se utilizaron contenedores de la STL más allá de QVector provisto por Qt. La gestión de memoria es manual mediante punteros crudos, sin uso de smart pointers. El juego está diseñado para resolución fija de 800×600 píxeles y no contempla redimensionamiento de ventana.

Usuarios destinatarios

El sistema está dirigido a un único tipo de usuario: el jugador, quien interactúa con el juego a través del teclado y el ratón sin necesidad de configuración previa ni conocimientos técnicos.

Metodología y planificación

El desarrollo de *Star Wars Tennis* siguió una metodología incremental dividida en dos momentos de entrega definidos por el curso, cada uno con un alcance funcional propio y completo.

El primer momento comprendió el diseño y la implementación del modo Tennis (vista cenital). En esta etapa se estableció la arquitectura general del sistema, se definió la jerarquía de clases base y se desarrollaron los

componentes fundamentales del juego: las clases Pelota, Jugador, Enemigo, Obstaculo, Marcador y FisicaMotor, junto con el ciclo de juego principal en MainWindow. Al finalizar este momento, el modo Tennis estaba completamente funcional, incluyendo el sistema de puntuación con reglas reales de tenis (*games, sets, deuce, ventaja y tie-break*), el obstáculo central (cinturón de asteroides) con desvío aleatorio, la IA del TIE Fighter y la selección de dificultad desde el menú principal.

El segundo momento se centró en la implementación del modo Jedi (vista lateral fija), que representó una expansión significativa sobre la base existente. En lugar de crear una arquitectura paralela, se optó por extender las clases ya construidas mediante un mecanismo de modo dual, de forma que Jugador y Enemigo pudieran comportarse de manera completamente distinta según el modo activo. En esta etapa se incorporaron: la física parabólica (con componentes horizontal y vertical independientes afectadas por la gravedad), el sistema de swing del sable de luz (movimiento circular alrededor del Jedi calculado *frame a frame*), los power-ups (incremento de daño y congelación) y penalidades (lentitud y velocidad extrema), el agente inteligente con aprendizaje (ciclo de percepción, razonamiento, acción y aprendizaje, almacenando errores en un `map<int,float>` memoria), el sistema de sonido (música de fondo y efectos de sonido diferenciados por modo), las animaciones del Jedi y los temporizadores adicionales requeridos por este modo (cuenta regresiva y duración de efectos temporales).

Este enfoque por momentos permitió que cada entrega fuera un producto funcional e independiente, facilitando la evaluación progresiva del proyecto

y la detección temprana de problemas de diseño antes de abordar la siguiente fase.

Diseño y arquitectura del sistema

Organización jerárquica de componentes

El sistema se estructura en torno a una clase controladora central, `MainWindow`, que hereda de `QMainWindow` y actúa como punto de coordinación de todos los elementos del juego. Esta clase gestiona el ciclo de actualización mediante un `QTimer` principal (configurado a 16 milisegundos para mantener 60 frames por segundo), administra los estados del juego a través del enumerado `ModoJuego` con tres valores posibles (`MODO_MENU`, `MODO_TENIS`, `MODO_JEDI`), y contiene referencias a todos los objetos activos en la escena.

La representación visual del juego se apoya en el sistema de escena de Qt, usando `QGraphicsScene` como contenedor de elementos y `QGraphicsView` como superficie de renderizado. Todos los objetos del juego que aparecen en pantalla heredan de `QGraphicsItem`, ya sea directamente o a través de la clase `Personaje`.

Jerarquía de clases

La clase `Personaje` hereda de `QGraphicsItem` y actúa como clase base que centraliza el estado compartido de los actores del juego: los atributos `ancho`, `alto` y `sprite`, accesibles mediante `getAncho()` y `getAlto()`. No define una

interfaz abstracta; sus subclases Jugador y Enemigo implementan por su cuenta los métodos requeridos por Qt: `boundingRect()`, `paint()` y `shape()`.

Jugador encapsula tanto al X-Wing del modo Tenis (vista cenital) como al Jedi del modo Jedi (vista lateral fija). El comportamiento correspondiente a cada modo se activa mediante el método `setModoJedi()`, que reconfigura los atributos de movimiento, los sprites, los límites de desplazamiento y la lógica de física asociada. Es `MainWindow` quien decide qué métodos de movimiento invocar según el modo activo y las teclas presionadas:

- En modo Tenis, llama a `moverArriba()` y `moverAbajo()` para desplazar el X-Wing verticalmente a lo largo del lado izquierdo de la pantalla.
- En modo Jedi, llama a `moverIzquierda()`, `moverDerecha()`, `saltar()` (con física de gravedad) y `activarSwing()` (movimiento circular alrededor del cuerpo del Jedi calculado *frame a frame*).

Enemigo cumple una función análoga: encapsula tanto al TIE Fighter del modo Tenis como al AT-AT del modo Jedi:

- En modo Tenis, sigue verticalmente la posición de la pelota con un margen de error configurable según la dificultad seleccionada.
- En modo Jedi, implementa un ciclo de agente inteligente compuesto por cuatro etapas: percepción del estado de la pelota mediante `leerPelota()` (observa posición, velocidad y trayectoria), razonamiento y selección de tipo de disparo mediante `procesar()` (predice la posición futura usando el modelo de movimiento parabólico y consulta la memoria de errores almacenada), ejecución

del disparo mediante `ejecutar()` (lanza la bola de fuego Sith hacia la posición calculada), y actualización de la memoria de aprendizaje mediante `aprender()` (guarda el error de predicción cometido en un `map<int,float>` memoria para corregir estimaciones futuras).

La clase Pelota hereda directamente de `QGraphicsItem` y soporta dos modos de movimiento:

- Rectilíneo uniforme para el modo Tennis, con rebote en bordes superiores e inferiores, y ángulo de salida variable según el punto de impacto contra la nave (calculado por `FisicaMotor`).
- Parabólico con gravedad para el modo Jedi, con componentes horizontal y vertical independientes determinadas por las ecuaciones de la cinemática de proyectil.

En el segundo modo incorpora lógica de rebote inelástico (coeficiente de restitución menor a uno, simulando pérdida de energía en cada rebote), contador de rebotes (máximo de dos toques en el suelo antes de que el Jedi pierda una vida), estado de congelación (activado por power-up, detiene la bola con vibración oscilatoria amortiguada modelada con una función sinusoidal) y marcado de si fue desviada por el jugador, lo cual determina si puede causar daño al AT-AT.

Obstaculo representa el campo de fuerza del modo Tennis, con movimiento vertical oscilante entre dos límites de pantalla (de arriba abajo). Al colisionar con el proyectil, introduce un desvío aleatorio controlado simulando una colisión elástica.

Marcador hereda de `QGraphicsTextItem` e implementa la lógica completa del sistema de puntuación del tenis, incluyendo la conversión de puntos al sistema 15-30-40, *deuce*, *ventaja* y *tie-break*.

FisicaMotor es una clase utilitaria que calcula el ángulo de salida del proyectil en función del punto de impacto sobre el personaje, generando trayectorias más cerradas o más abiertas según qué parte de la nave golpeó el torpedo.

Las clases Dificultad, SonidoManager, MenuDificultad y PantallaInstrucciones son clases de soporte que no participan en la jerarquía de herencia principal:

- Dificultad centraliza todos los parámetros numéricos del juego según el nivel seleccionado (velocidad de la pelota, margen de error de la IA, tiempo límite, duración de power-ups y penalidades, etc.).
- SonidoManager gestiona la música de fondo mediante `QMediaPlayer` y los efectos de sonido mediante instancias de `QSoundEffect` (golpes, puntos, swing del sable, power-ups, penalidades, victoria y derrota).
- MenuDificultad y PantallaInstrucciones construyen y controlan los elementos visuales de la interfaz de navegación.

Dependencias externas

El proyecto depende exclusivamente del framework Qt 6, utilizando los módulos Qt Widgets para la interfaz gráfica y el sistema de escena, y Qt Multimedia para la reproducción de audio. El sistema de construcción

empleado es CMake 3.16. No se utilizan bibliotecas externas adicionales.

La gestión de memoria es manual mediante punteros crudos, sin uso de *smart pointers*, y los contenedores de la STL no se emplean más allá de QVector provisto por Qt.

Procedimientos de prueba

Métodos de verificación

La verificación del juego se realizó mediante pruebas manuales funcionales, ejecutando el programa en Qt Creator y evaluando el comportamiento del sistema ante distintos escenarios de juego. Dado que el proyecto no implementa un framework de pruebas automatizadas, cada funcionalidad fue validada de forma directa por los desarrolladores durante y después de su implementación.

Casos de prueba

El primer conjunto de pruebas se enfocó en el modo Tenis (vista cenital). Se verificó que:

- La pelota rebotara correctamente en los bordes superior e inferior de la pantalla.
- El ángulo de salida variara según el punto de impacto en el personaje (generando trayectorias más cerradas o más abiertas).
- El campo de fuerza central (cinturón de asteroides) introdujera desvíos aleatorios en la trayectoria al colisionar con el proyectil.

- El sistema de puntuación registrara correctamente los puntos, *games*, *sets*, *deuce*, *ventaja* y *tie-break*.
- Al alcanzar un ganador, la escena se limpiara y el juego regresara al menú tras mostrar la pantalla de resultado.

El segundo conjunto de pruebas cubrió el modo Jedi (vista lateral fija). Se verificó que:

- La bola de fuego Sith siguiera una trayectoria parabólica afectada por la gravedad (con componentes horizontal y vertical independientes).
- Rebota una vez en el suelo con pérdida de energía (coeficiente de restitución menor a uno) y fuera eliminada al segundo contacto.
- El swing del sable de luz solo deflectara la bola cuando la colisión ocurriera durante el intervalo activo (es decir, cuando la pelota se encontrara dentro del arco de rotación del sable en el momento exacto del *swing*).
- Únicamente las bolas marcadas como desviadas (`setDesviada(true)`) pudieran dañar al AT-AT.
- La barra de vida del AT-AT se actualizara correctamente con cada impacto, cambiando de color de verde a amarillo y de amarillo a rojo según el porcentaje de vida restante.

El tercer conjunto de pruebas evaluó los power-ups y penalidades. Se verificó que:

- El multiplicador de daño se aplicara correctamente durante el tiempo activo (diez segundos) y se revirtiera al expirar.

- La congelación detuviera las bolas sin eliminarlas (entrando en estado de vibración oscilatoria amortiguada) y las reactivara al terminar.
- La penalidad de lentitud redujera la velocidad de movimiento del Jedi a la mitad y se revirtiera tras cinco segundos.
- La penalidad de velocidad extrema incrementara la velocidad de las bolas activas en un 50%.

El cuarto conjunto de pruebas verificó las transiciones entre estados. Se comprobó que:

- Al presionar Escape durante cualquier modo, el juego regresara correctamente al menú sin dejar *timers* activos ni objetos huérfanos en memoria.
- La música de fondo cambiara correctamente entre el menú y el modo Tenis
- Los *flags* de control victoriaFlag y gameOverFlag evitaran la ejecución duplicada de las funciones de fin de partida (victoria(), gameOver()).

El quinto conjunto de pruebas evaluó el comportamiento del agente inteligente a lo largo de partidas extendidas, verificando que:

- El AT-AT diversificara sus tipos de disparo con el tiempo (mediante el sistema de pesos almacenado en memoria).
- La dificultad percibida aumentara progresivamente en los niveles Normal y Difícil, a medida que el agente reducía su error de predicción intercambio tras intercambio.

Resultados obtenidos

Todas las funcionalidades principales del juego operaron según lo esperado durante las pruebas. Se identificaron y corrigieron varios comportamientos no deseados durante el desarrollo, entre ellos:

- La posibilidad de que bolas no desviadas dañaran al AT-AT (corregido mediante el sistema de marcado `setDesviada(true)`).
- La ejecución duplicada de `victoria()` y `gameOver()` ante condiciones de carrera entre *timers* (corregido mediante los *flags* de control).
- Desvíos excesivos en el ángulo de disparo del AT-AT que generaban bolas inalcanzables para el jugador (corregido ajustando los rangos de variación angular según la dificultad).

Revisión de calidad

El código fue revisado de forma iterativa por ambos integrantes del equipo, aplicando correcciones incrementales confirmadas mediante pruebas de ejecución directa en Qt Creator. Cada corrección significativa fue registrada en el repositorio de GitHub con mensajes de *commit* descriptivos, permitiendo un seguimiento claro de la evolución del proyecto y facilitando la identificación de regresiones.

Guía de instalación y uso

Requisitos técnicos

Para compilar y ejecutar *Star Wars Tennis* se requiere tener instalado Qt 6

con los módulos Qt Widgets y Qt Multimedia, CMake versión 3.16 o superior, y un compilador compatible con el estándar C++17. El proyecto fue desarrollado y probado en Qt Creator. Se recomienda usar la misma versión del IDE para garantizar compatibilidad con la configuración del archivo CMakeLists.txt.

Instrucciones de instalación y compilación

El código fuente del proyecto se encuentra disponible en el repositorio de GitHub del equipo. Para obtenerlo, se debe clonar el repositorio ejecutando el siguiente comando en una terminal: `git clone`

`https://github.com/mateoarbelaezc-web/MOMENTO1.git`

Una vez clonado, se abre Qt Creator, se selecciona la opción de abrir proyecto y se carga el archivo CMakeLists.txt ubicado en la raíz del repositorio. Qt Creator configurará automáticamente el proyecto mediante CMake. Finalmente se compila y ejecuta el proyecto desde el botón de ejecución del IDE.

Controles

En el modo Tenis (vista cenital), el jugador mueve su nave (X-Wing) con las teclas de dirección Arriba y Abajo.

En el modo Jedi (vista lateral fija), el jugador controla a Luke Skywalker mediante los siguientes comandos:

- A y D: movimiento horizontal por el suelo.
- Barra espaciadora: salto (con física de gravedad).

- J: activación del swing del sable de luz (movimiento circular alrededor del cuerpo del Jedi).

En ambos modos, presionar la tecla Escape regresa al menú principal.

La selección de modo y dificultad se realiza desde el menú principal haciendo clic con el ratón sobre los botones correspondientes:

- Modo Tenis: dificultad Fácil o Difícil.
- Modo Jedi: tres niveles de dificultad (Fácil, Normal, Difícil).

Desde el menú también es posible consultar la pantalla de instrucciones, que resume los controles y objetivos de cada modo.

Recomendaciones

Se recomienda ejecutar el juego en una pantalla con resolución mínima de 800×600 píxeles, ya que la ventana del juego tiene un tamaño fijo de 806×606 píxeles y no contempla redimensionamiento (tal como se especifica en la sección de Restricciones). Para una experiencia óptima se sugiere jugar con el sonido activado, ya que los efectos de audio proporcionan retroalimentación importante sobre eventos del juego como golpes, power-ups, resultados de partida y transiciones entre estados.

Resultados y discusión

Funcionamiento logrado

El resultado final del proyecto es un videojuego completamente funcional con dos modos de juego diferenciados, menú principal interactivo, sistema de sonido, pantalla de instrucciones y transiciones entre estados:

- El modo Tenis (vista cenital) implementa las reglas completas del tenis con *deuce*, *ventaja* y *tie-break*, dos niveles de dificultad (Fácil y Difícil) y un campo de fuerza central (cinturón de asteroides) que introduce variabilidad en las trayectorias mediante desvíos aleatorios controlados.
- El modo Jedi (vista lateral fija) implementa física parabólica (con componentes horizontal y vertical independientes afectadas por la gravedad), sistema de vidas (máximo de dos rebotes en el suelo antes de perder una vida), temporizador de cuenta regresiva (que añade presión al jugador), power-ups y penalidades (incremento de daño, congelación, lentitud y velocidad extrema), animaciones del personaje (swing del sable de luz modelado como rotación con ángulo variable) y un agente inteligente con capacidad de aprendizaje progresivo (ciclo de percepción, razonamiento, acción y aprendizaje, almacenando errores en un `map<int,float>` memoria).

Ambos modos comparten una arquitectura de clases unificada que demuestra en la práctica los principios de reutilización y extensibilidad propios de la programación orientada a objetos, permitiendo que Jugador y Enemigo se comporten de manera completamente distinta según el modo activo mediante el mecanismo de modo dual.

Limitaciones actuales

La gestión de memoria se realizó íntegramente mediante punteros crudos sin uso de *smart pointers*, lo que implica que un manejo incorrecto de las transiciones entre estados podría derivar en fugas de memoria. Si bien se implementaron mecanismos de limpieza explícita en `volverAlMenu()`, `victoria()` y `gameOver()` (eliminando objetos dinámicos y deteniendo temporizadores), la ausencia de herramientas de análisis de memoria como Valgrind impide garantizar que no existan fugas residuales en todos los escenarios posibles.

El juego está diseñado para una resolución fija de 800×600 píxeles (ventana de 806×606 píxeles) y no contempla adaptación a distintos tamaños de pantalla. Asimismo, el modo Jedi permite únicamente una bola activa a la vez, lo que limita la intensidad del juego en los niveles de mayor dificultad.

El agente inteligente del AT-AT, aunque funcional y progresivo, basa su aprendizaje en un conjunto fijo de cuatro tipos de disparo predefinidos. Esto significa que su capacidad de adaptación tiene un techo natural, y en partidas muy largas el jugador experimentado puede identificar y explotar los patrones del agente.

Dificultades enfrentadas

Una de las principales dificultades durante el desarrollo fue la sincronización de múltiples temporizadores en el modo Jedi. La coexistencia del *timer* principal de 16 milisegundos, el temporizador de cuenta regresiva, el temporizador de disparos del AT-AT y el temporizador

de generación de power-ups generó condiciones de carrera que en algunos casos provocaban la ejecución duplicada de las funciones de victoria() y gameOver(). Esto se resolvió mediante el uso de *flags* booleanos victoriaFlag y gameOverFlag que bloquean la ejecución repetida de dichas funciones una vez que el estado final de la partida ha sido determinado.

Otra dificultad significativa fue la extensión de las clases Jugador y Enemigo para soportar dos modos de comportamiento completamente distintos (movimiento vertical con teclas direccionales en modo Tenis vs. movimiento horizontal, salto y swing en modo Jedi) sin duplicar código. La solución adoptada mediante el mecanismo de modo dual (activado por setModoJedi() y consultado mediante *flags* internos) resultó efectiva, aunque introdujo cierta complejidad interna en ambas clases, requiriendo que MainWindow decidiera qué métodos de movimiento invocar según el modo activo.

Mejoras planeadas

Como mejoras futuras se identifican:

- La posibilidad de permitir múltiples bolas simultáneas en el modo Jedi para aumentar la dificultad y la intensidad del juego.
- La incorporación de un sistema de puntuación con tabla de récords (top 5 de puntuaciones más altas, persistente entre sesiones).
- La adaptación de la interfaz a distintas resoluciones de pantalla (escalado dinámico de sprites y reconfiguración de la escena).

Conclusiones

El desarrollo de *Star Wars Tennis* permitió consolidar en un único proyecto los tres pilares fundamentales del curso: la programación orientada a objetos, el manejo de memoria dinámica y el diseño de interfaces gráficas con Qt. La implementación de dos modos de juego con mecánicas distintas (modo Tenis con vista cenital estilo *Pong*, y modo Jedi con vista lateral y física parabólica) sobre una arquitectura compartida demostró en la práctica el valor de conceptos como la herencia, el polimorfismo y el encapsulamiento, que durante el semestre habían sido trabajados principalmente en contextos teóricos o ejercicios aislados.

La decisión de extender las clases existentes mediante un mecanismo de modo dual (activado por `setModoJedi()` y gestionado por `MainWindow`) en lugar de crear jerarquías paralelas fue uno de los aprendizajes de diseño más significativos del proyecto. Aunque implicó mayor complejidad interna en las clases `Jugador` y `Enemigo`, permitió reutilizar una base de código ya probada y mantener la coherencia arquitectónica del sistema a lo largo de ambas fases de desarrollo (primer momento: modo Tenis; segundo momento: expansión al modo Jedi).

La implementación del agente inteligente del AT-AT representó el reto algorítmico más exigente del proyecto. Diseñar un ciclo de percepción, razonamiento, acción y aprendizaje (con sus correspondientes métodos `leerPelota()`, `procesar()`, `ejecutar()` y `aprender()`) que funcionara

correctamente en tiempo real, sin interferir con el ciclo de actualización principal de 16 milisegundos, requirió comprender con profundidad la sincronización de eventos en Qt y la gestión del estado del juego entre *frames* consecutivos. El almacenamiento del error de predicción en un `map<int,float>` memoria y su uso para corregir estimaciones futuras demostró cómo un mecanismo de aprendizaje simple puede generar un comportamiento adaptativo progresivo.

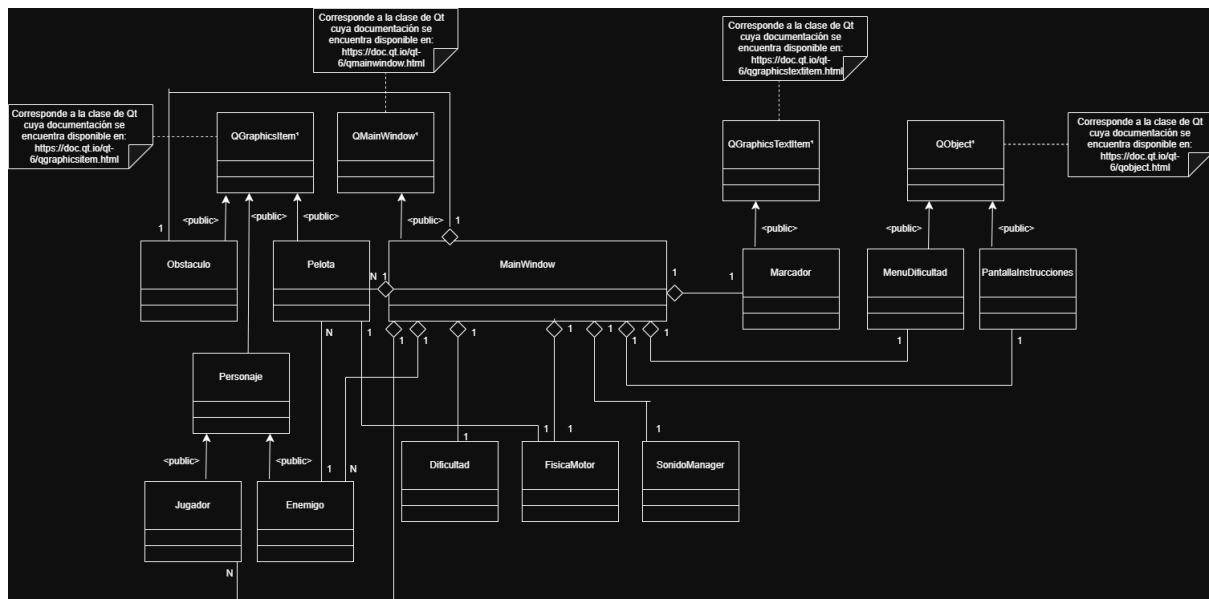
La gestión de múltiples temporizadores simultáneos en el modo Jedi (temporizador principal de actualización, temporizador de cuenta regresiva, temporizador de disparos del AT-AT y temporizador de generación de power-ups) evidenció que el desarrollo de software en tiempo real introduce una categoría de problemas distinta a la programación secuencial, donde el orden y la concurrencia de eventos pueden producir comportamientos inesperados difíciles de reproducir y depurar. Las soluciones adoptadas (uso de *flags* booleanos `victoriaFlag` y `gameOverFlag` para evitar ejecución duplicada, limpieza explícita de temporizadores en `volverAlMenu()`, `victoria()` y `gameOver()`), aunque funcionales, sugieren que en proyectos de mayor escala sería conveniente adoptar un modelo de gestión de estado más formal, como una máquina de estados finitos explícita.

En términos generales, el proyecto cumplió con los objetivos planteados y produjo un resultado funcional, interactivo y visualmente atractivo. La experiencia de desarrollar un producto completo con fecha de entrega definida, que incluye diseño, implementación, pruebas y documentación, constituyó un aprendizaje integral que trasciende los contenidos técnicos del

curso y anticipa las dinámicas del desarrollo de software profesional.

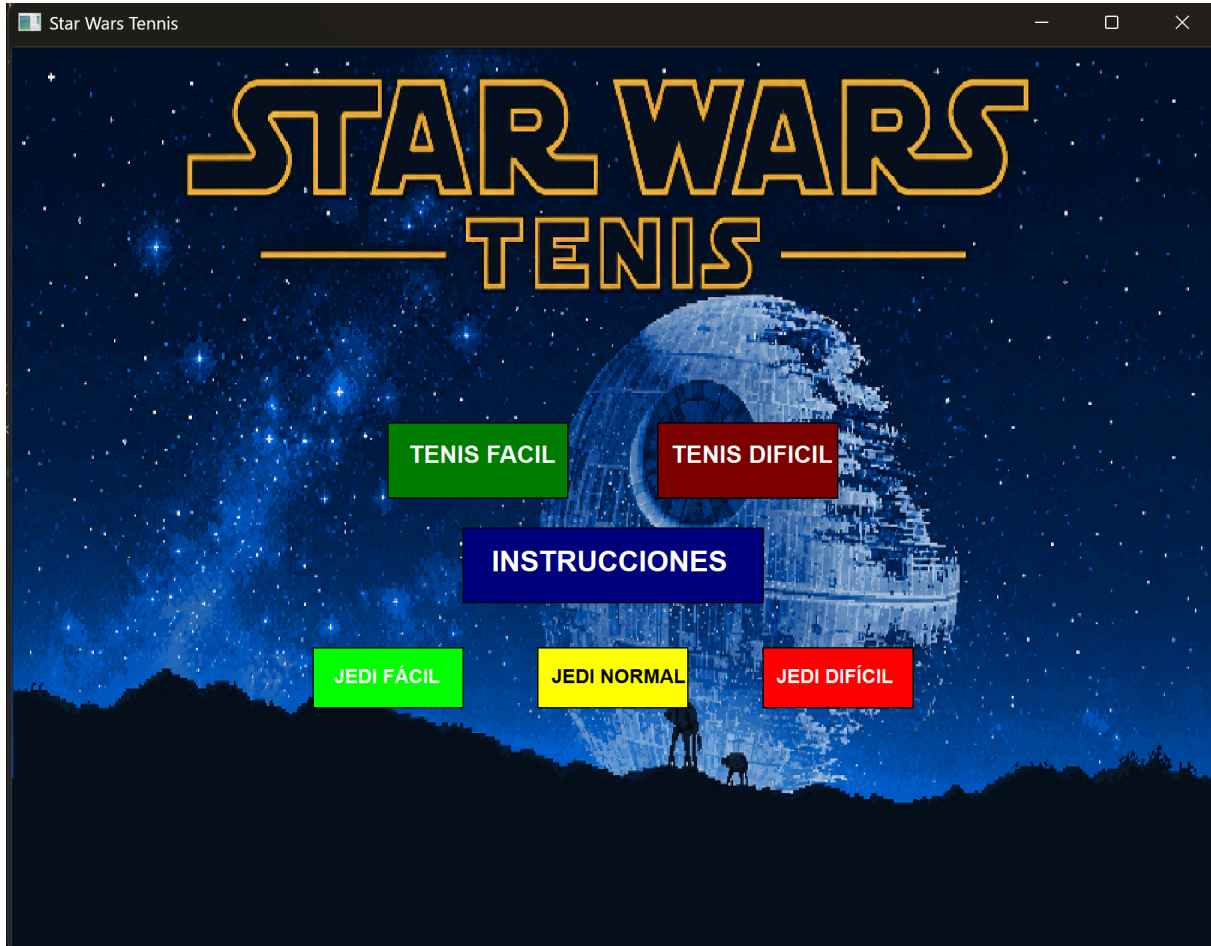
Anexos

Diagrama de clases



Capturas de pantalla del juego

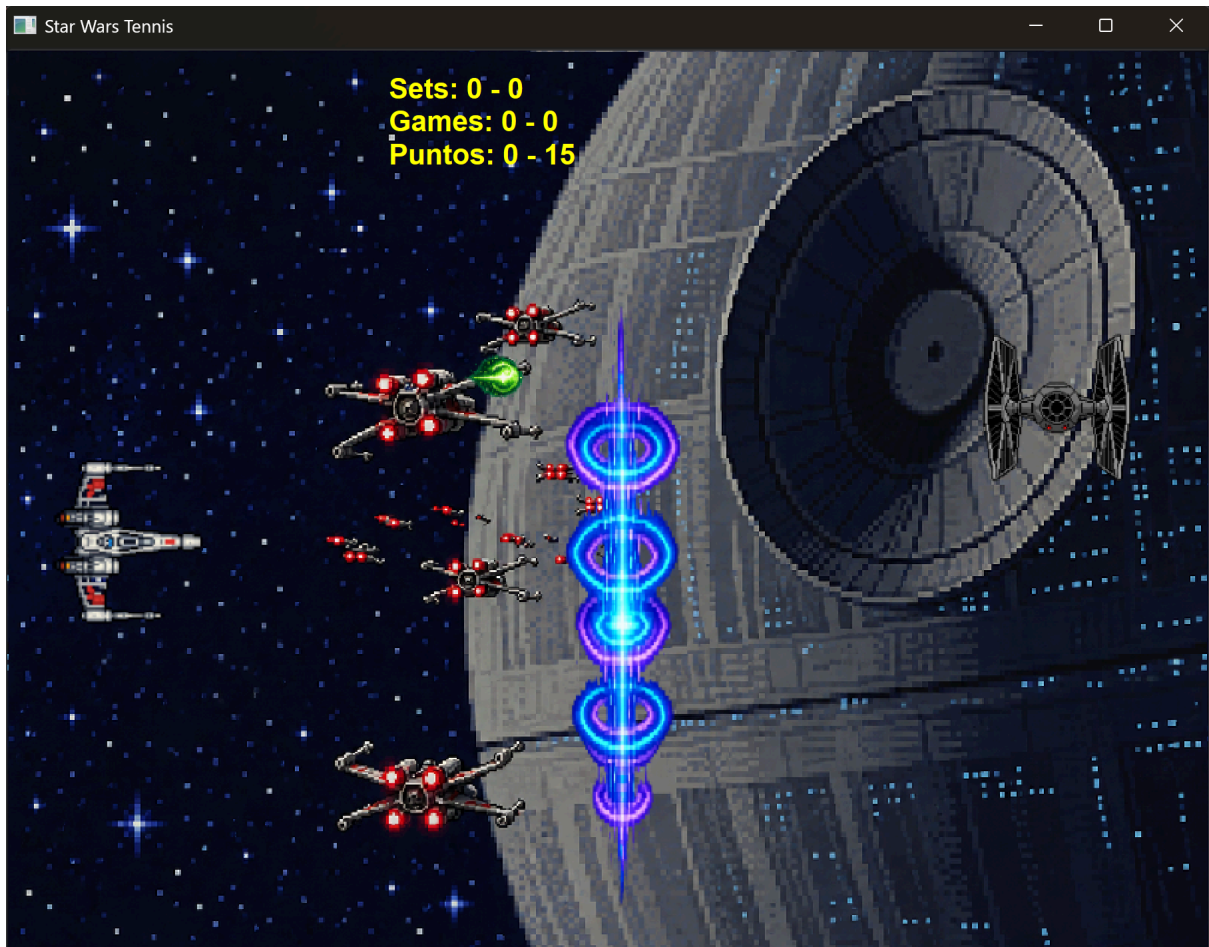
Pantalla de inicio



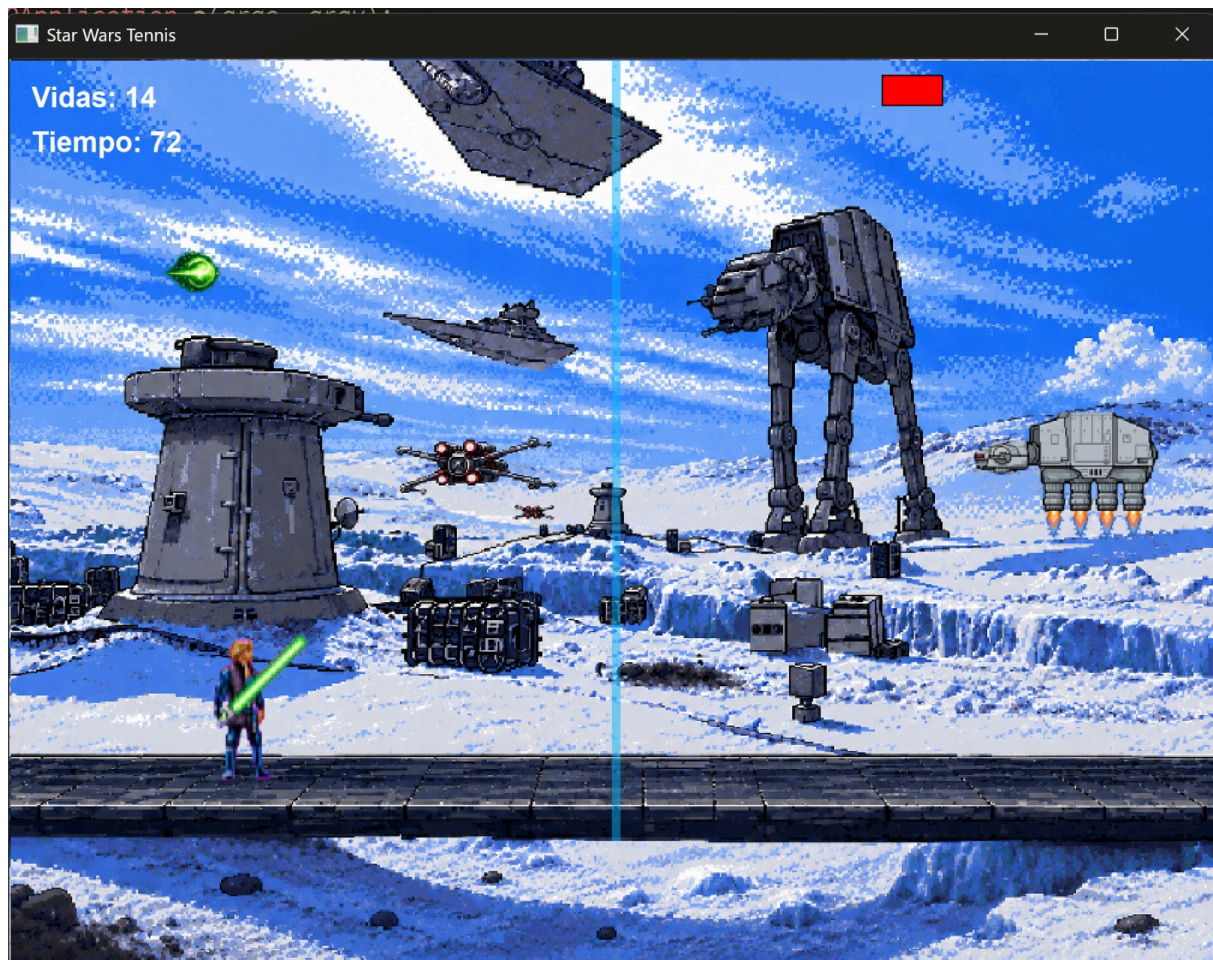
Instrucciones



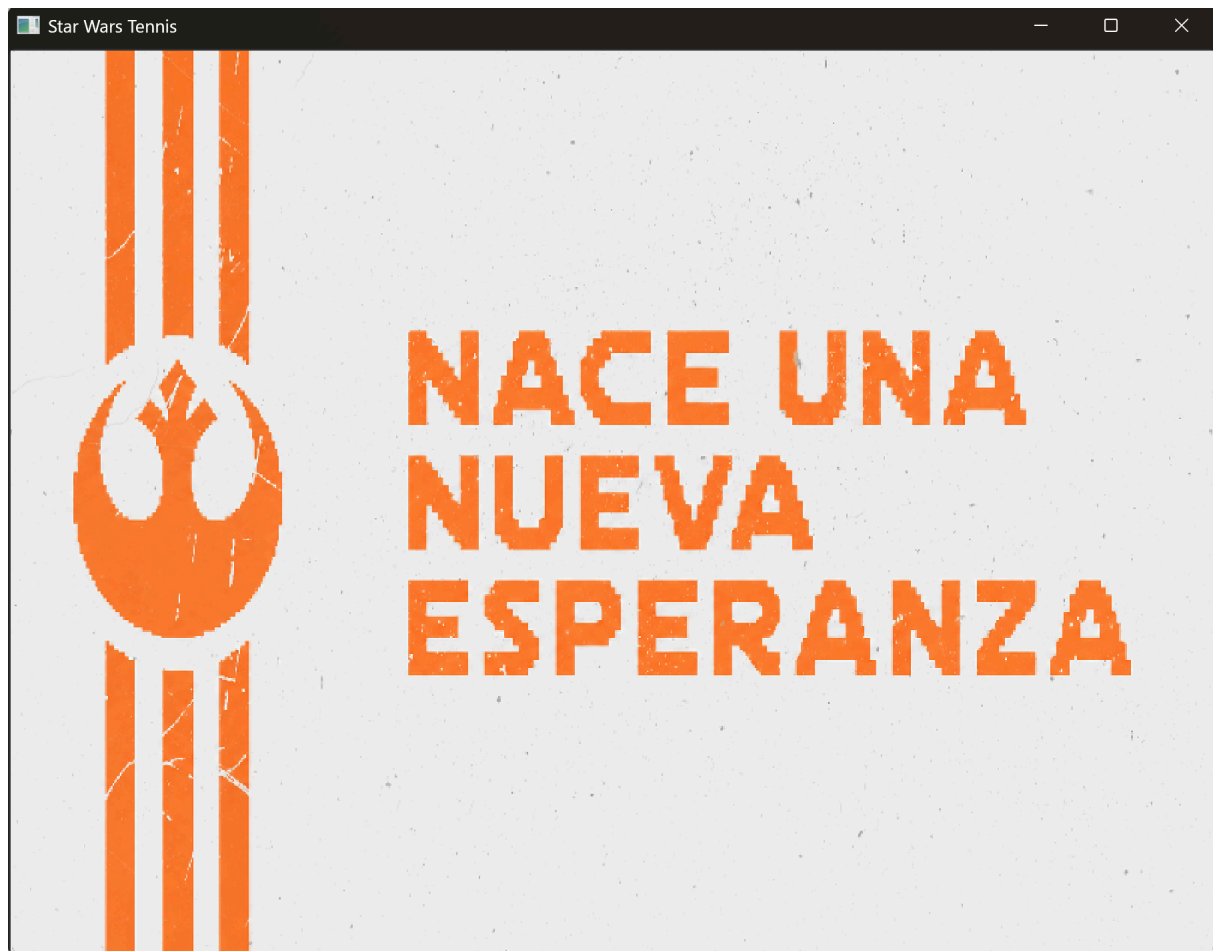
Modo Tenis



Modo Jedi



Pantalla de victoria



Pantalla de derrota



Fragmentos de código importantes

Cálculo del ángulo de impacto (FisicaMotor)

Este método calcula el ángulo de salida del proyectil en función del punto de impacto relativo sobre el personaje. Un impacto en el centro produce una trayectoria horizontal, mientras que los impactos en los extremos generan ángulos de hasta 45 grados.

```
void FisicaMotor::aplicarAnguloImpacto(Pelota* pelota, float yNave,
                                       float altoNave, float velocidad) {
    float puntoImpacto = (pelota->y() - yNave) / altoNave;
    puntoImpacto = qBound(-0.8f, puntoImpacto, 0.8f);
    float angulo = puntoImpacto * 45.0f * M_PI / 180.0f;
    float dirX = (pelota->getVelX() > 0) ? 1.0f : -1.0f;
    pelota->setVelX(dirX * velocidad * cos(angulo));
    pelota->setVelY(velocidad * sin(angulo));
}
```

Ciclo del agente inteligente (Enemigo)

Este fragmento muestra las cuatro etapas del ciclo del agente: percepción del estado de la pelota, selección ponderada del tipo de disparo, ejecución del disparo y actualización de la memoria de aprendizaje.

```

void Enemigo::procesar() {
    if (!modoJedi) return;
    if (!bolaEntrante) {
        float pesoTotal = 0;
        for (auto& rec : memoria)
            pesoTotal += rec.usos + rec exitos * dificultadIA;
        float r = (rand() % 10000) / 10000.0f * pesoTotal;
        float acum = 0;
        for (auto& rec : memoria) {
            acum += rec.usos + rec exitos * dificultadIA;
            if (r <= acum) { tipoSeleccionado = rec.tipo; break; }
        }
    } else {
        switch (ultimaZonaImpacto) {
            case 1: tipoSeleccionado = 1; break;
            case 2: tipoSeleccionado = 0; break;
            default: tipoSeleccionado = 2; break;
        }
    }
}

```

Física parabólica con rebote (Pelota)

Este método actualiza la posición de la pelota aplicando gravedad frame a frame, gestiona el rebote inelástico en el suelo y determina cuándo la pelota debe ser eliminada de la escena.

```
void Pelota::actualizarConGravedad(float gravedad, float sueloY) {  
    if (!usarGravedad || frozen || dead) return;  
    velY += gravedad;  
    setPos(x() + velX, y() + velY);  
    if (y() + radio * 2 >= sueloY) {  
        setY(sueloY - radio * 2);  
        velY = -velY * restitution;  
        bounceCount++;  
        if (bounceCount >= 2) dead = true;  
    }  
}
```